Jane decided to implement the form component using React. Specifically, she planned to use the `useActionState` hook (https://react.dev/reference/react/useActionState) for streamlined form handling.

```
const [state, formAction, isPending] = useActionState(fn, initial
```

This hook, introduced in React 19, manages the full lifecycle of form submissions by returning a `state` variable (initialized to `initialState` and later updated with the action's return value), providing a modified action to pass to `<form action>` or `<button formAction>`, and exposing an `isPending` flag for loading states.

Given the typical 45-minute coding window in a one-hour interview, Jane focused on these core elements:

- Implement the basic form structure with `name` and `email` inputs.
- Set up validation rules for required fields and restrictions on the `name` field.
- Leverage native email string validation.
- Include a hidden input field for the `access_key`, ideally secure it by hiding the actual key from the code.
- Integrate the `useActionState` hook for efficient form handling.

By focusing on these essentials and clearly communicating her enhancement plans, Jane demonstrated her problem-solving skills and ability to work within time constraints.

Listing 2.8 is her first draft of the implementation, defined in `components/form/FormSimple.tsx`.

Listing 2.8 Simple form implementation

```
const Form: React.FC = () => {
  const [state, formAction, isPending] = useActionState<FormState
    async (prevState, formData: FormData) => { #B
      try {
        const response = await fetch('https://api.web3forms.com/s
          method: 'POST',
          body: formData as BodyInit, #D
```